FleetZero

FleetPulse & BusPulse Development Portfolio

Role: Full-Stack Developer
Duration: Jan. 2026 – Apr. 2026
Company: FleetZero Inc. | Toronto, ON | Hybrid | Part-Time

Technologies: Angular 19, TypeScript, HTML, CSS, Angular Material, RxJS, C#, ASP.NET Core, REST APIs, Swagger/OpenAPI, ApexCharts, Karma, Jasmine, Git, Azure DevOps, CI/CD, Agile/Scrum, Kanban

Prepared by
Jaturaput (Mac) Jongsubcharoen

Telephone: +1 (647) 425-9756
Email: macker.jong@gmail.com
LinkedIn: www.linkedin.com/in/jaturaput-jongsubcharoen
GitHub: https://github.com/Jaturaput-Jongsubcharoen

Project Overview

As a Full-Stack Developer at FleetZero, I contributed to the development and maintenance of FleetPulse and BusPulse, enterprise fleet management applications supporting vehicle, facility, inspection, ticket, project, and maintenance operations.

My work covered frontend and backend development, including Angular 19, TypeScript, RxJS, C#, ASP.NET Core, REST APIs, and Swagger/OpenAPI. I developed and enhanced dashboards and operational workflows, integrated frontend functionality with backend APIs, implemented interactive ApexCharts visualizations, optimized API request handling and application performance, resolved production and dependency issues, and improved responsive user interfaces across client and administrator views.

I also developed and maintained Karma/Jasmine unit tests, worked with mocked services and API responses, resolved test failures, and contributed to CI/CD validation and Azure DevOps workflows. Development was completed through team-based Git workflows, branches, pull requests, code reviews, Azure DevOps Boards, sprint/Kanban workflows, and Agile collaboration.

Project Objectives

The project aimed at:

- Develop and maintain scalable **FleetPulse and BusPulse** fleet management applications.
- Build operational workflows for **vehicles, facilities, inspectors, inspections, projects, tickets, and maintenance data**.
- Develop frontend functionality with **Angular, TypeScript, and RxJS** and integrate it with **C# / ASP.NET Core REST APIs**.
- Test and troubleshoot API communication using **Swagger/OpenAPI** and application-level API integration.
- Create interactive dashboards and data visualizations using **ApexCharts**.
- Support **client, administrator, and role-based application experiences**.
- Optimize API requests, data processing, dashboard loading, and overall application performance.
- Maintain application reliability through **Karma/Jasmine automated testing and CI/CD validation**.
- Deliver responsive, maintainable, and production-ready enterprise functionality through **Git, Azure DevOps, and Agile development workflows**.

My Contributions

During my time at FleetZero, I contributed across multiple areas of the application, including:

- **Full-stack development:** Angular 19, TypeScript, RxJS, C#, ASP.NET Core, REST APIs, and Swagger/OpenAPI.
- **FleetPulse development:** initial Angular UI setup, facility and garage interfaces, Vehicle Management, vehicle search, drag-and-drop workflows, dashboard components, and supporting unit tests.
- **BusPulse Client UI:** /client routing, dashboards, navigation, reports, authentication/guards, role-based navigation, and reuse of shared application components.

- **Dashboard development:** Client and Admin dashboard functionality, widgets, filtering, responsive/fullscreen behavior, and interactive ApexCharts visualizations.
- **REST API integration:** project, vehicle, ticket, dashboard, inspector, and inspection-related data flows, including filter-dependent API requests and frontend/backend data communication.
- **Inspector Management:** Card/Table views, inspector profiles, View/Edit Profile workflows, Add Inspector functionality, forms, validation, and API-integrated inspector management.
- **Inspection workflows:** hierarchical client/project/vehicle selection and retrieval of vehicle-specific inspection tasks through interconnected API data.
- **Vehicle Station Tracking:** timeline rendering, API data parsing, detailed/merged views, station-type grouping, tooltips, scrolling, filtering, and usability improvements.
- **Data visualization:** Tickets by Status, Overall/Repeated Defects by Area, Project Comparison, Project Status, Propulsion Types, Safety Critical Defects, and other operational visualizations.
- **Performance optimization:** reduced duplicate/redundant REST API requests, eliminated unnecessary per-project API fan-out, and improved RxJS request coordination, caching/deduplication, and refresh behavior.
- **API data processing:** normalization, aggregation, percentage calculations, fallback handling, ticket totals, asset totals, and project/vehicle-specific dashboard calculations.
- **Debugging and maintenance:** resolved production bugs, ApexCharts rendering problems, Angular dependency conflicts, widget visibility issues, API-data issues, and responsive UI problems.
- **Dependency management:** maintained Angular 19 compatibility across Angular Material, CDK, Localize, ng-apexcharts, and conflicting third-party packages.
- **Git collaboration:** feature branches, merges, pull requests, code reviews, conflict/dependency resolution, and integration with the team's main development branch.
- **UI/UX:** responsive layouts, light/dark themes, filtering, pagination, tooltips, chart readability, fullscreen behavior, navigation, and usability improvements.

The following sections present **selected development highlights** from my FleetPulse and BusPulse contributions, with screenshots and technical explanations demonstrating the implementation and results.

Technology Validation

Technology	Evidence / Example
Angular	Dashboard, inspection UI, inspector flow; Merge PRs and file-level diffs in Angular components
TypeScript	Service logic and component implementations; dashboard-projects.service.ts
RxJS	forkJoin, switchMap, shareReplay, retry, finalize, catchError, timer; dashboard-projects.service.ts and dashboard files
Angular Material	App uses Angular Material globally; Repo-wide usage, not uniquely attributable
HTML	Inspection list and dashboard templates; Component HTML files in related commits
CSS/SCSS	Dashboard widget styling and panel layout; Dashboard SCSS and feature changes
C#	Backend REST API development, debugging, and frontend-backend integration
ASP.NET Core	Backend REST API development, controllers/services, and Angular integration
REST APIs	/Tickets, /tickets/dashboard, /StationTrackers, /Projects, /projects/{projectId}/vehicles; Service files and commit messages
Swagger/OpenAPI	API endpoint documentation, testing, debugging, and frontend-backend integration
ApexCharts	Treemap, station tracking, comparison charts; Commit messages + chart utility files
Karma	Unit testing, test debugging, and resolving failing Angular tests
Jasmine	Test fixes and spec changes; Spec files
Git	Local Git history/branches; git log and branches
Azure DevOps	Kanban boards, sprint tasks/issues, Git branches, pull requests, code reviews, CI validation, and team development workflows
CI/CD	Fixed failing tests required by CI; 9162c68a and related test fixes

Verified Azure DevOps Repository Contributions

The following table highlights my merged pull requests and code contributions from the FleetZero Azure DevOps repository, covering feature development, REST API integration, performance optimization, debugging, testing, and UI/UX improvements across FleetPulse and BusPulse.

Date	Commit	Author Email	Message
2026-04-24	9162c68a	Jaturaput@fleetzero.ai	Merged PR 369: feature/748: Fix failing Karma/Jasmine unit tests after main merge
2026-04-24	9a5a87d8	Jaturaput@fleetzero.ai	Merged PR 365: fix(inspection-list): make Filter Scope and Search toolbar sticky below app navbar
2026-04-23	1fe9976c	Jaturaput@fleetzero.ai	Merged PR 364: feat(inspection-list): add client-side pagination to all drill-down levels
2026-04-22	f40e0178	Jaturaput@fleetzero.ai	Merged PR 362: feat(inspection-list): implement Add New Inspection drawer with filter sync and project-scoped category validation
2026-04-20	70d49f87	Jaturaput@fleetzero.ai	Merged PR 355: Inspection List: Complete Task Details Columns, ProjectType-Based Result Binding, and Save/Revert Workflow Improvements
2026-04-17	f5dedda4	Jaturaput@fleetzero.ai	Merged PR 354: Inspection List Table Foundation: Project/Vehicle Context, Real Dataset Filters, and Project-Scoped Task Rendering
2026-04-16	3c1ddc5e	Jaturaput@fleetzero.ai	Merged PR 353: Create Inspection List page UI under BusPulse Simulator with initial table and filter layout
2026-04-15	ee63b06f	Jaturaput@fleetzero.ai	Merged PR 350: Inspector Management End-to-End Improvements: List

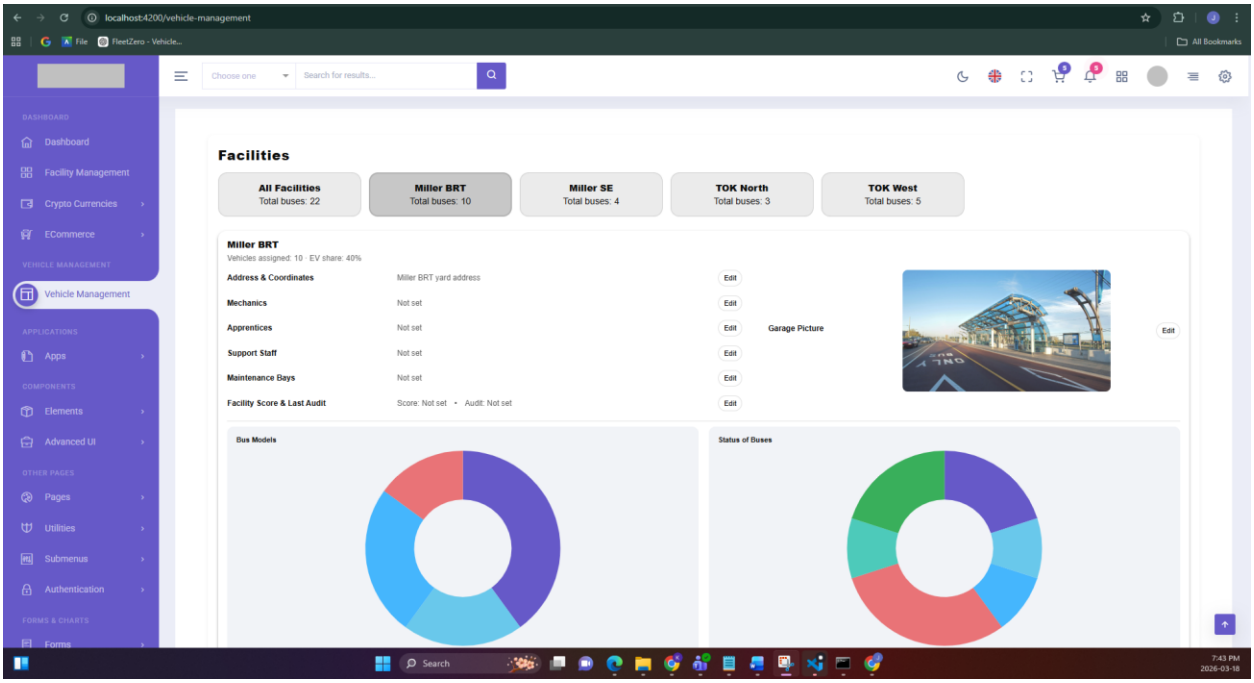
			Sorting, Add/Profile Data Completeness, and User Certification Sync Contract Fix
2026-04-14	2ded2748	Jaturaput@fleetzero.ai	Merged PR 348: Fix dashboard request amplification by cleaning default filter params in /api/projects and hardening widget refresh flows
2026-04-13	463c410a	Jaturaput@fleetzero.ai	Merged PR 345: Reduce Tickets API fan-out and duplicate requests in dashboard widgets
2026-04-09	0917fe2a	Jaturaput@fleetzero.ai	Merged PR 344: Reduce /api/tickets/dashboard fan-out on Dashboard load and keep explicit filter behavior
2026-04-09	ee3fdc75	Jaturaput@fleetzero.ai	Merged PR 335: Reduce StationTrackers request fan-out and eliminate duplicate widget re-triggers in Dashboard
2026-04-08	2e795864	Jaturaput@fleetzero.ai	Merged PR 332: Update Inspector Add/Edit flows for real API integration, profile fields, and uniqueness validation
2026-04-03	50eb6838	Jaturaput@fleetzero.ai	Merged PR 328: Add Inspector end-to-end stepper flow, API wiring, and live inspector list refresh
2026-04-02	b40e4f94	Jaturaput@fleetzero.ai	Merged PR 327: Refactor Add New Inspector UI into 3-Step Flow with Basic Info Save and Inline Validation
2026-04-01	5d45b4cc	Jaturaput@fleetzero.ai	Merged PR 325: Implement Add Inspector Basic Info Flow and Organization Dropdown
2026-03-28	f8f10671	Jaturaput@fleetzero.ai	Merged PR 321: Dashboard: Split Type of Time into Project and Vehicle widgets with

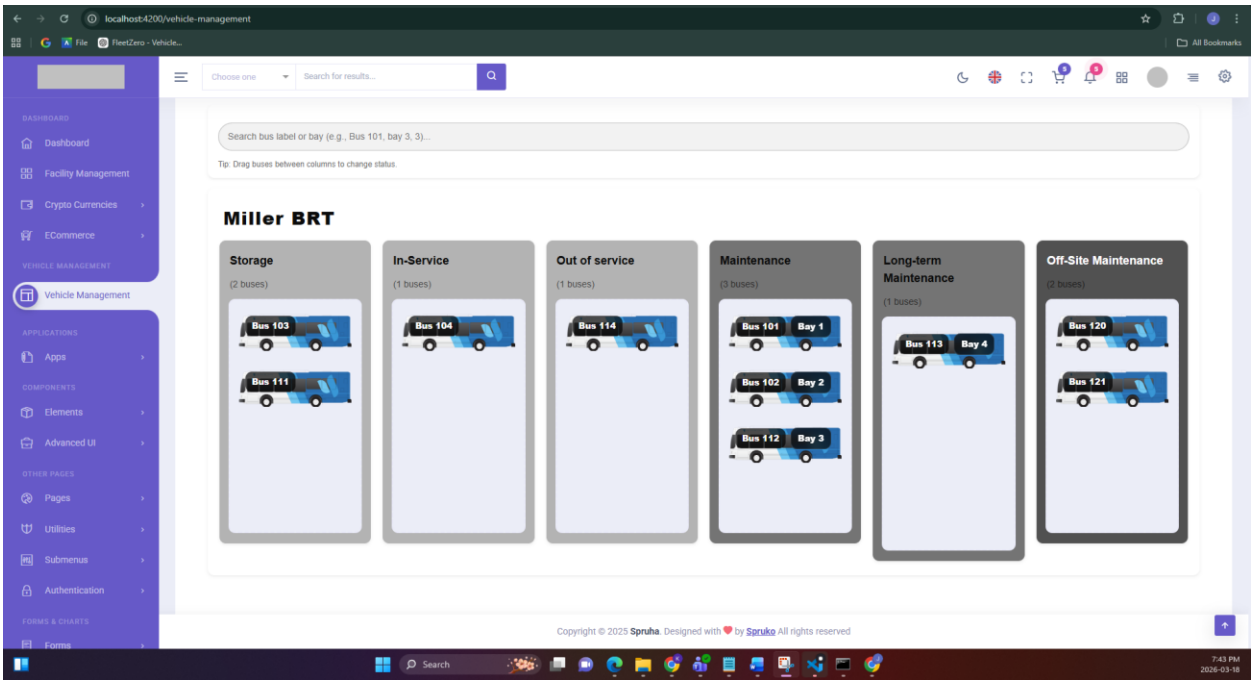
			admin-only visibility and performance guardrails
2026-03-27	3aeccc51	Jaturaput@fleetzero.ai	Merged PR 320: Projects Comparison by Area (Total Defects): migrate to tickets-dashboard heatmap, project-type coloring, dual-axis labels, and fullscreen/UI parity
2026-03-26	6de23552	Jaturaput@fleetzero.ai	Merged PR 318: feat(#532): scope Snags project and vehicle filters to client account (match Dashboard behavior)
2026-03-25	b72c9343	Jaturaput@fleetzero.ai	Merged PR 317: Refine Station Comparison Charts: Use Average Duration per Vehicle, Enhance Tooltips, and Add X/Y Scrolling for Readability
2026-03-24	e69e0122	Jaturaput@fleetzero.ai	Merged PR 315: Enhance Average Type of Time Comparison by Project Widgets
2026-03-24	146fec66	Jaturaput@fleetzero.ai	Merged PR 313: Projects Comparison by Station Type: rename widget and replace static heatmap with paged StationTrackers aggregation
2026-03-20	38b7b2a5	Jaturaput@fleetzero.ai	Merged PR 312: Improve Vehicle Station Tracking usability (paging, labels, visibility, and fullscreen consistency)
2026-03-17	830ba8cc	Jaturaput@fleetzero.ai	Merged PR 310: Remove time display from Start and End dates in Vehicle Station Tracking tooltip
2026-03-16	51a3196a	Jaturaput@fleetzero.ai	Merged PR 308: #511 Fix Client Dashboard widget display logic and Repeated Defects gauge behavior
2026-03-13	828e5934	Jaturaput@fleetzero.ai	Merged PR 307: Vehicle Station Tracking Widget:

			Render Fix, Performance Optimization, and UX Enhancements
2026-03-11	6b452097	Jaturaput@fleetzero.ai	Merged PR 298: Widget: implement defects-by-area treemap with API + Production placeholder
2026-03-04	0231140e	Jaturaput@fleetzero.ai	Merged PR 296: feat(dashboard): integrate Tickets by Status widget with API, canonicalize status labels, and unify client/admin dashboards
2026-03-02	06cbc910	Jaturaput@fleetzero.ai	Merged PR 293: feat(dashboard): derive total tickets and total assets from API
2026-02-26	ec477e51	Jaturaput@fleetzero.ai	Merged PR 291: feat(dashboard): show treemap values by default for overall and repeated defects by area
2026-02-23	1dbd58ee	Jaturaput@fleetzero.ai	Merged PR 289: feat(client-dashboard): use shared dashboard ticket APIs for project/vehicle-filtered totals
2026-02-20	47b7b4f8	Jaturaput@fleetzero.ai	Merged PR 288: fix: align Angular 19 dependencies
2026-02-16	242f36b7	Jaturaput@fleetzero.ai	Merged PR 284: Task 437: Client dashboard API integration and UI enhancements
2026-02-02	038ad5d4	Jaturaput@fleetzero.ai	Merged PR 283: Client Dashboard - Hide charts for individual project & UI improvements
2026-01-28	94cbc927	Jaturaput@fleetzero.ai	Merged PR 280: Client UI: /client routing, dashboard, navigation and client reports

1. FleetPulse Project – Selected Development Highlights

1.1. Setup FleetPulse Angular UI (initial code and assets)





Set up and developed the initial **FleetPulse Angular UI** based on the Spruha template, including the dashboard structure, application environment, styling, facility assets, and reusable **Facility and Garage components**. Built the **Vehicle Management dashboard** to display facility information, vehicle totals, garage details, maintenance bays, bus-model and vehicle-status visualizations, and facility-specific operational data.

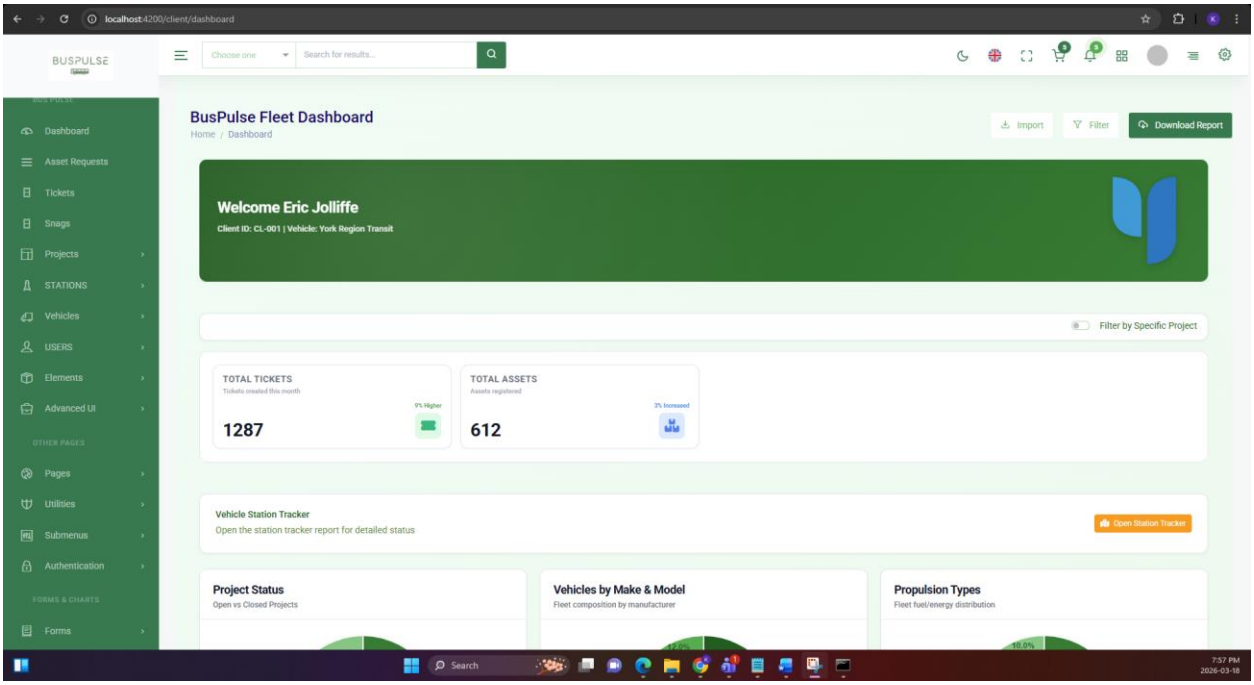
Implemented an interactive vehicle management interface with **drag-and-drop functionality** for moving buses between operational categories such as **Storage, In-Service, Out of Service, Maintenance, Long-**

term Maintenance, and Off-Site Maintenance. Added **vehicle search, horizontally scrollable facility columns, garage/bay visualization, facility selection, responsive UI behavior, and theme-mode support** to improve usability when managing larger vehicle inventories.

1.2. Add unit tests for Vehicle Management UI components to make to ensure the code has no error.
Developed **Karma/Jasmine unit tests** for the FleetPulse Vehicle Management interface to validate the functionality and stability of the newly developed components. Testing covered **VehicleManagementComponent** functionality including **drag-and-drop interactions, vehicle search and search results, facility/garage behavior, and bay modal interactions**.
Added test coverage for supporting Facility components including **FacilityButtons, FacilityColumns, FacilityBay, FacilitySearch, FacilitySearchResult, and FacilityStrips**, using mocked dependencies and a **mocked Chart.js component** where required. These tests helped verify component behavior, detect regressions, and ensure the Vehicle Management functionality remained stable as the FleetPulse application evolved.

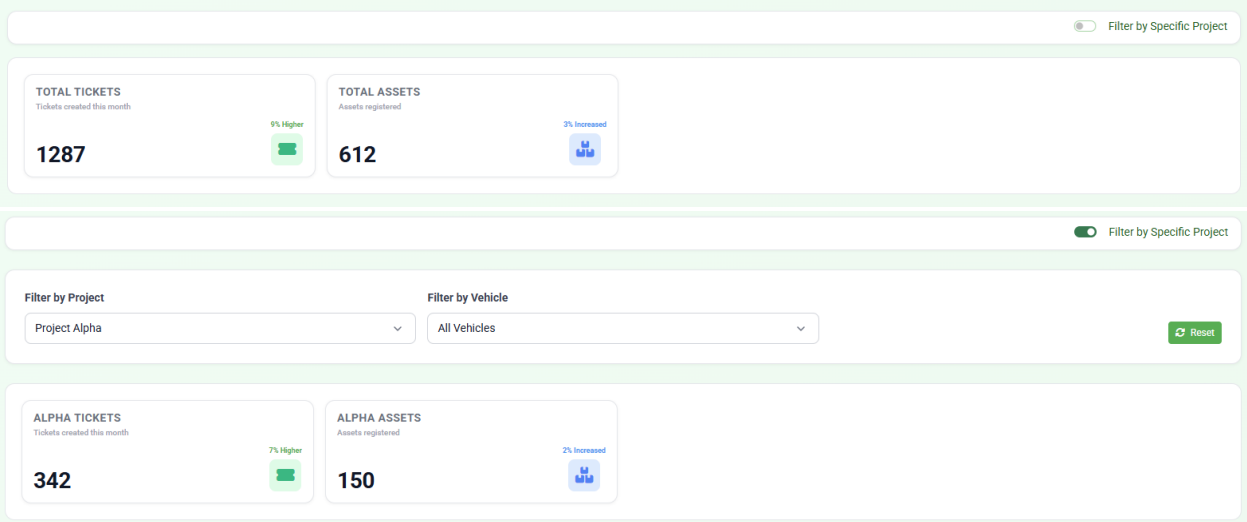
2. BusPulse Project – Selected Development Highlights

2.1. Set up Client UI: /client routing, dashboard, navigation and client reports



Developed the initial **BusPulse Client Portal** under the /client route, establishing the foundation for the client-facing application with **Angular routing, dashboard structure, client data handling, authentication/authorization, navigation, and reporting functionality**. Integrated the Client Portal with the existing application architecture while maintaining separation between **Client and Admin views**. Reused existing **shared Angular components and services** without unnecessary duplication, including **full/content layouts, page headers, header/footer, side navigation, switcher, notification sidebar, six dashboard chart components, six report components and report services, authentication guards, role-based guards, and navigation services**. Configured these shared resources within the new Client Portal to support **role-based routing and consistent dashboard functionality**.

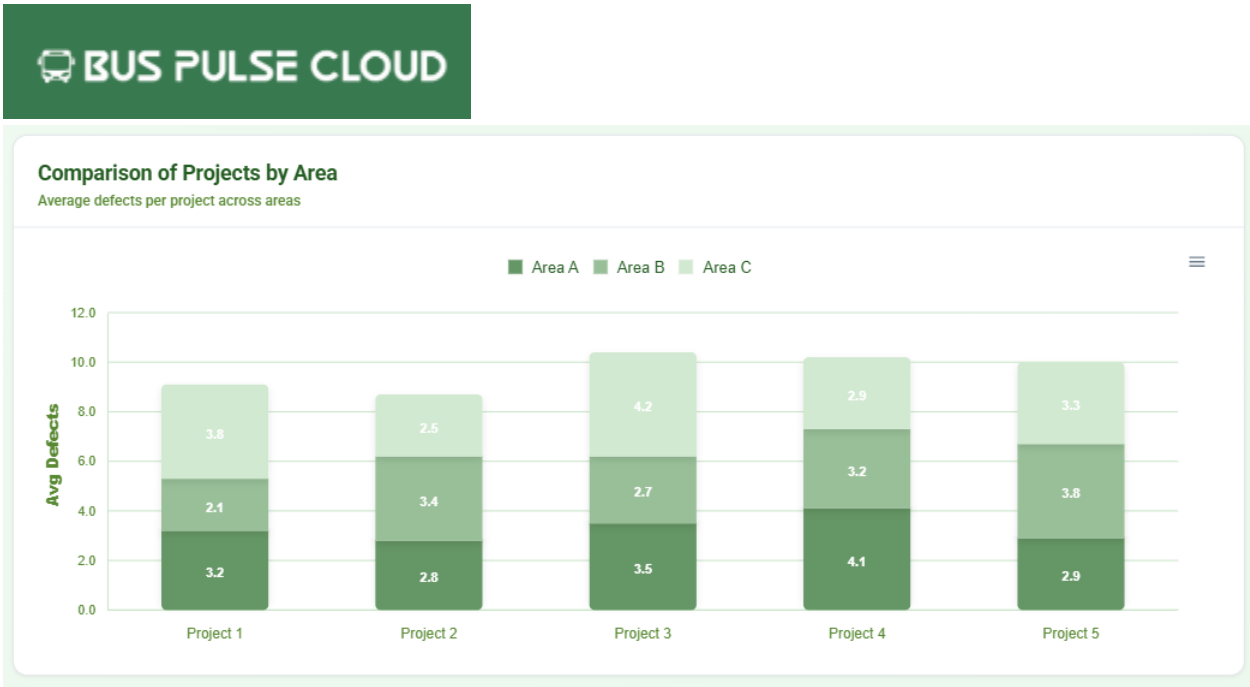
2.2. Improve Project Filtering, Chart Visibility, and Dashboard UI

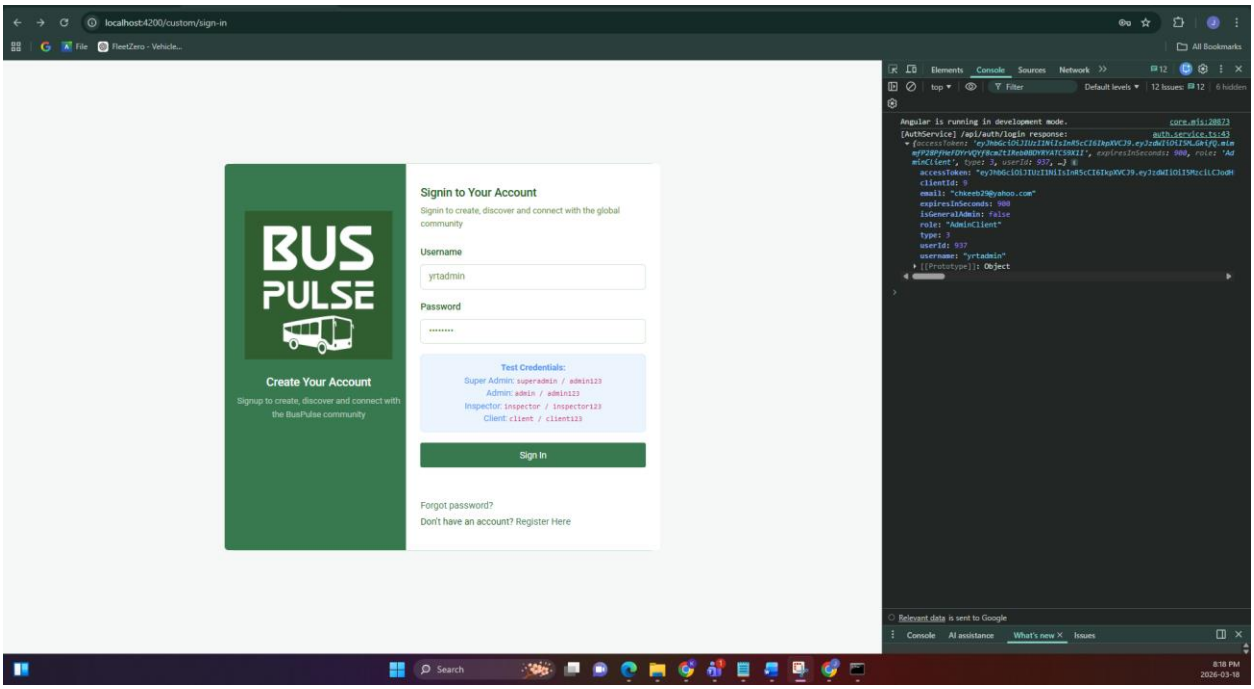


Enhanced the **Client Dashboard filtering workflow** by implementing the **Filter by Specific Project** functionality, allowing users to switch from an overall fleet view to project-specific views using **Project and Vehicle filters**. Updated dashboard summary information so **ticket and asset totals respond to the selected filtering scope**, while adjusting widget visibility so users see dashboard information relevant to their current selection.

Resolved **ApexCharts console and rendering errors** when switching between filtered and unfiltered dashboard states, improved chart and widget styling, and implemented consistent **light/dark theme support** throughout the Client Dashboard. These improvements strengthened the dashboard's **stability, usability, responsive behavior, and visual consistency**.

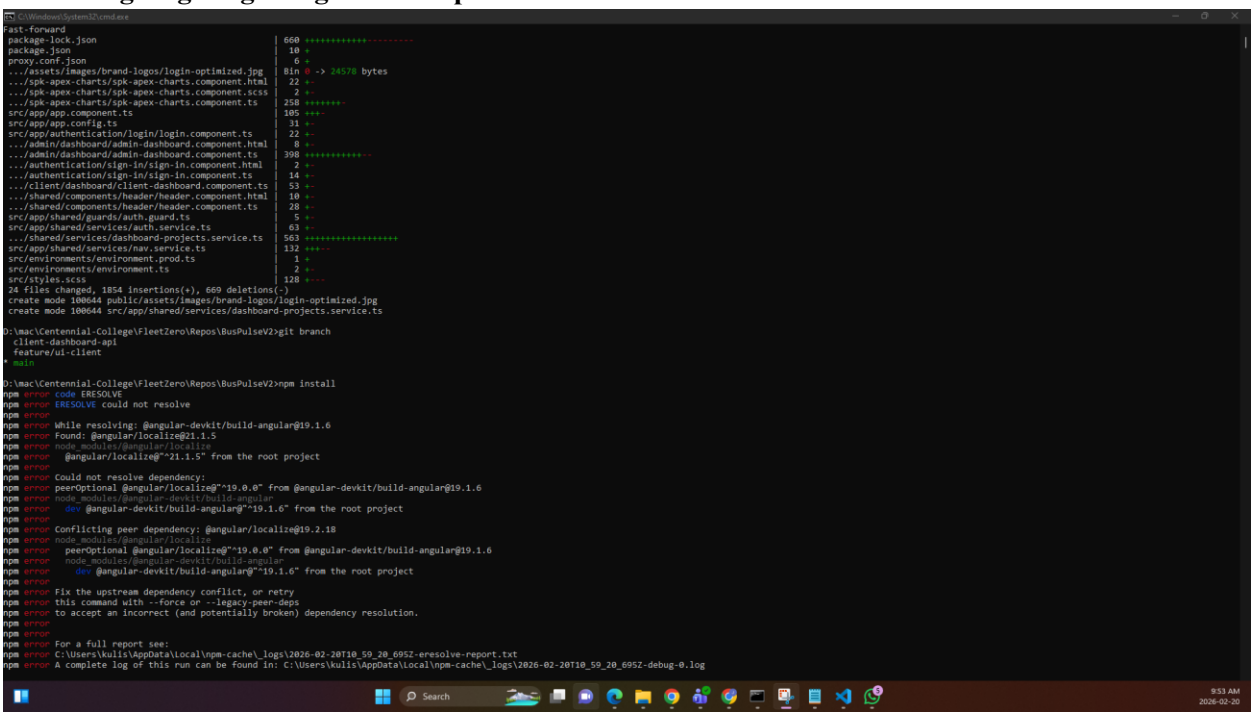
2.3. Enhance Client dashboard API integration and UI





Enhanced the **Client Dashboard UI and chart readability** by updating the **Bus Pulse Cloud logo** from a dark green background to a **transparent background**, allowing it to integrate more consistently across the application's layouts and themes. Improved the **Comparison of Projects by Area** stacked bar chart by enabling **always-visible data labels**, allowing users to immediately view and compare defect values for each project and inspection area without hovering, while preserving **interactive tooltips** for additional chart details.

2.4. Fixing Bug: align Angular 19 dependencies



Resolved **Angular 19 dependency conflicts** that prevented npm install from completing after integrating the latest main changes. Removed **@kolkov/angular-editor** because of its incompatible Angular 20+ peer dependency, pinned **ng-apexcharts to 1.15.0** for Angular 19 compatibility, and aligned **Angular Material, CDK, and Localize to 19.2.18**. These changes restored dependency compatibility and kept the project aligned with the existing **Angular 19** application environment.

2.5. Use shared dashboard ticket APIs for project/vehicle-filtered totals

Updated the **Client Dashboard API integration** to dynamically retrieve ticket totals from the shared **/api/tickets/dashboard REST API** based on the active project and vehicle filters. When a project is selected with **All Vehicles**, the dashboard retrieves the total tickets for that project; when a specific vehicle is selected, the request is scoped to the selected **project and vehicle**. Integrated **/api/projects/{projectId}/vehicles** to dynamically load the corresponding **fleet numbers** for each selected project, keeping vehicle filtering synchronized with the dashboard data.

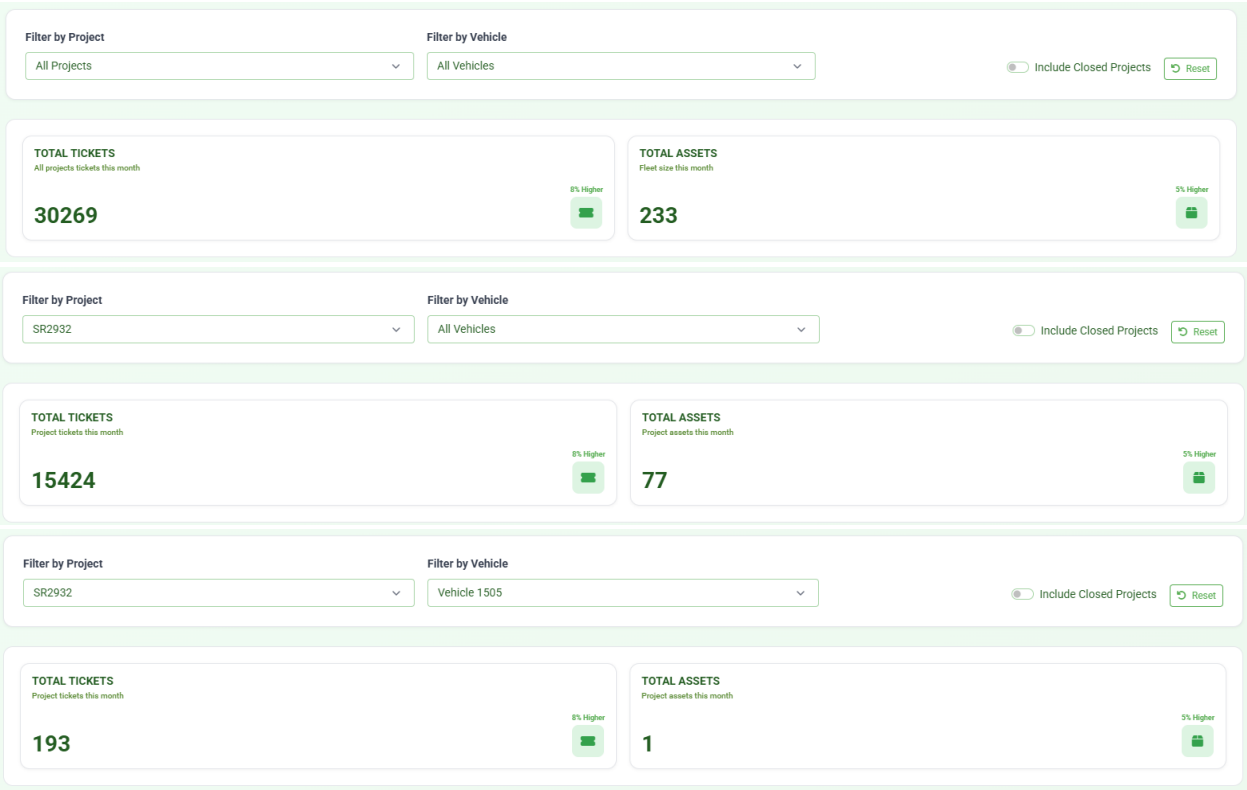
Enhanced the dashboard visualizations by increasing the size and thickness of the **Project Status, Propulsion Types, Repeated Defects, and Safety Critical Defects** charts, making percentages and values easier to read. Updated the widgets to use consistent **green color shades** and improved the overall chart presentation to create a clearer and more cohesive dashboard UI.

2.6. Show treemap values by default for overall and repeated defects by area

Updated the **Client Dashboard REST API integration** to use the shared **/api/tickets/dashboard endpoint** for dynamically retrieving ticket totals based on active project and vehicle filters. Implemented logic so selecting a **project with All Vehicles** returns the project's total tickets, while selecting a **specific vehicle** returns ticket data scoped to both the project and vehicle. Integrated **/api/projects/{projectId}/vehicles** to dynamically retrieve and populate the available fleetNumbers whenever a project is selected.

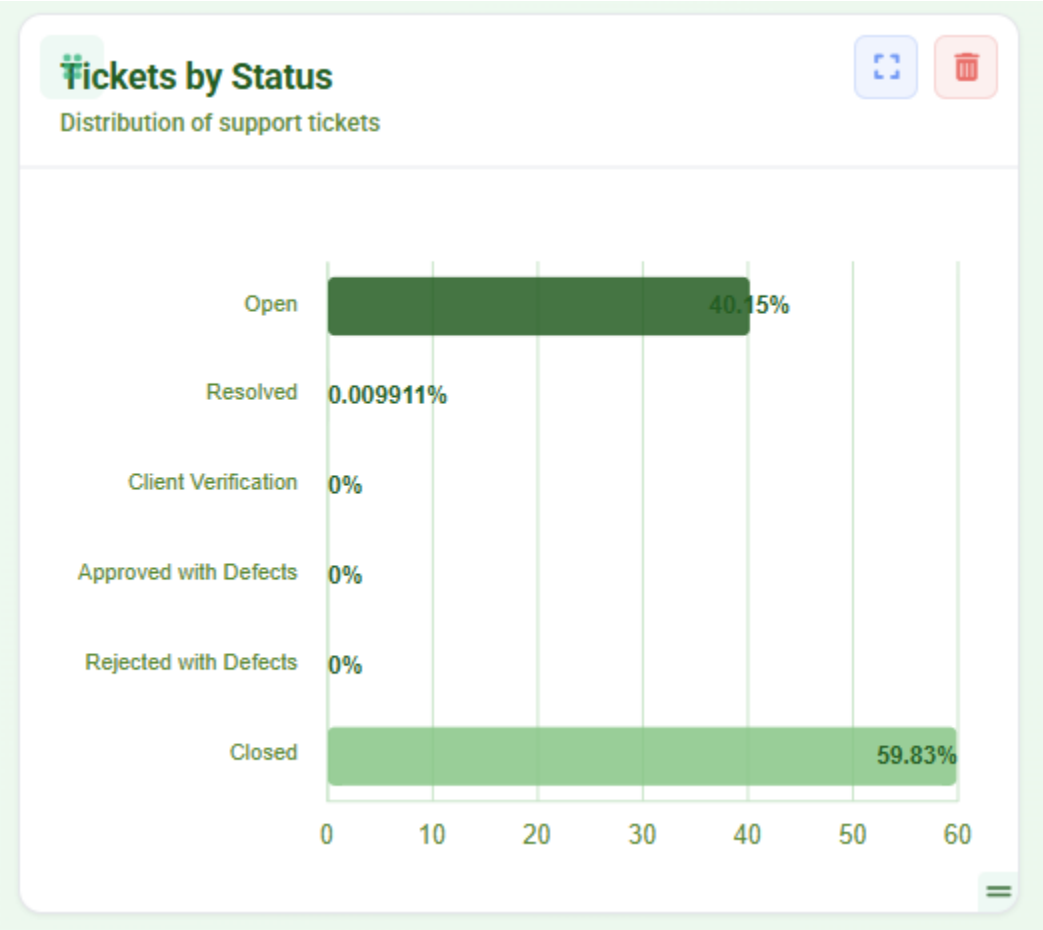
Improved the dashboard UI by increasing the visual thickness of the **Project Status, Propulsion Types, Repeated Defects, and Safety Critical Defects** widgets for better readability and updated their visual styling with **green color shades** to provide a more consistent dashboard appearance.

2.7. Deriving total tickets and total assets, as combining everything into a single file causes the API fetching to break.



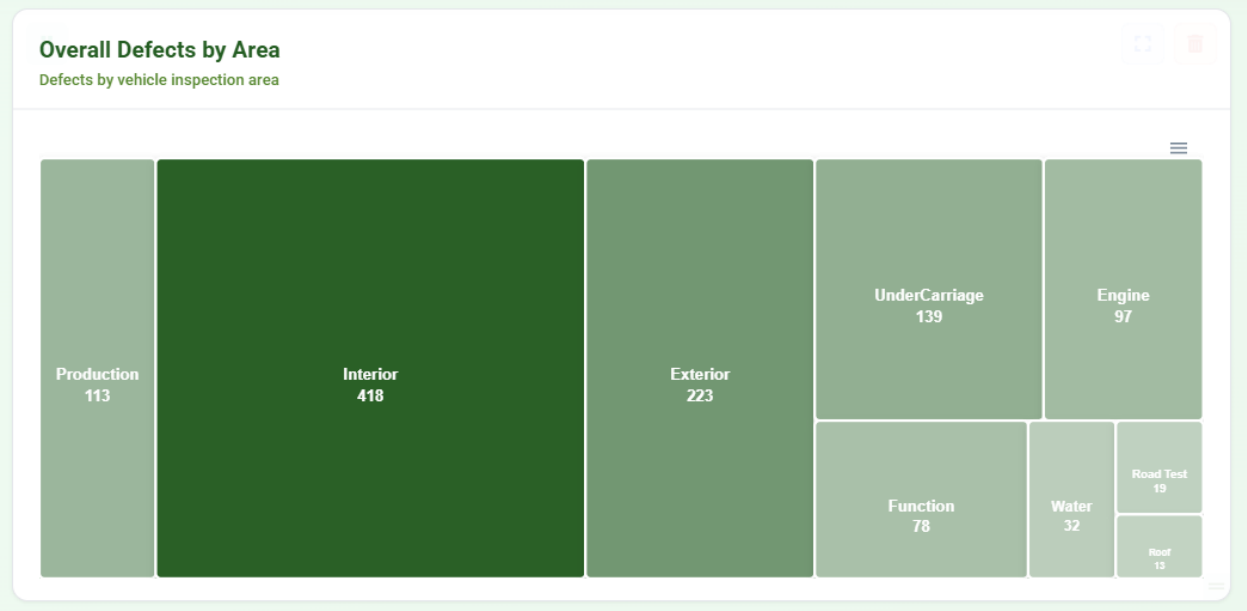
Updated the **Client Dashboard data-processing logic** in refreshClientView() to dynamically retrieve ticket totals for different **project and vehicle filter combinations**, including single-project, all-project aggregation, and vehicle-specific queries. Improved **Total Assets calculation** so the dashboard correctly returns **1 for a specific vehicle, the project vehicle count for a selected project with All Vehicles, the complete client vehicle count for All Projects and All Vehicles, and 1 when a specific vehicle is selected across projects**.

2.8. feat(dashboard): integrate Tickets by Status widget with API, canonicalize status labels, and unify client/admin dashboards (refs #450)



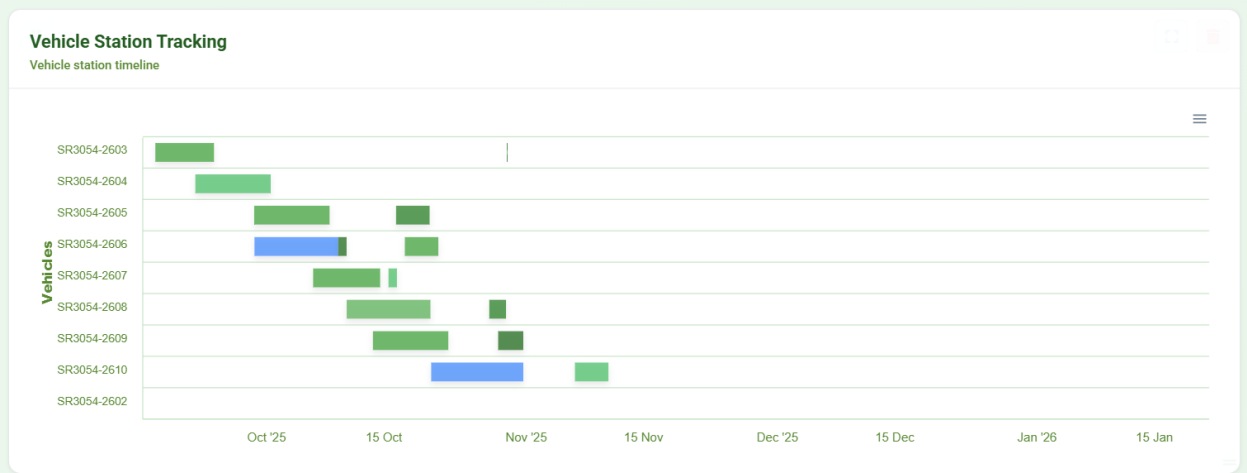
Replaced **demo and hardcoded Tickets by Status data** with authoritative values retrieved from the **/api/tickets/dashboard REST API**. Implemented **status normalization, percentage distribution calculations, and enhanced chart tooltips** to ensure accurate and consistent ticket-status visualization across both **Client and Admin dashboards**.

2.9. Widget: implement defects-by-area treemap with API + Production placeholder



Connected the **defects-by-area treemap** to the existing **GET /api/tickets/dashboard REST API**, using the overallByArea response to dynamically display inspection areas including **Interior, Exterior, Engine, Roof, Function, Water, Road Test, and UnderCarriage**, with each treemap block sized according to its **defect count**. Because the API did not yet return defect data for **Production**, implemented a temporary **Production placeholder (~10%)** to visualize the expected dashboard layout while preserving integration with the available real API data.

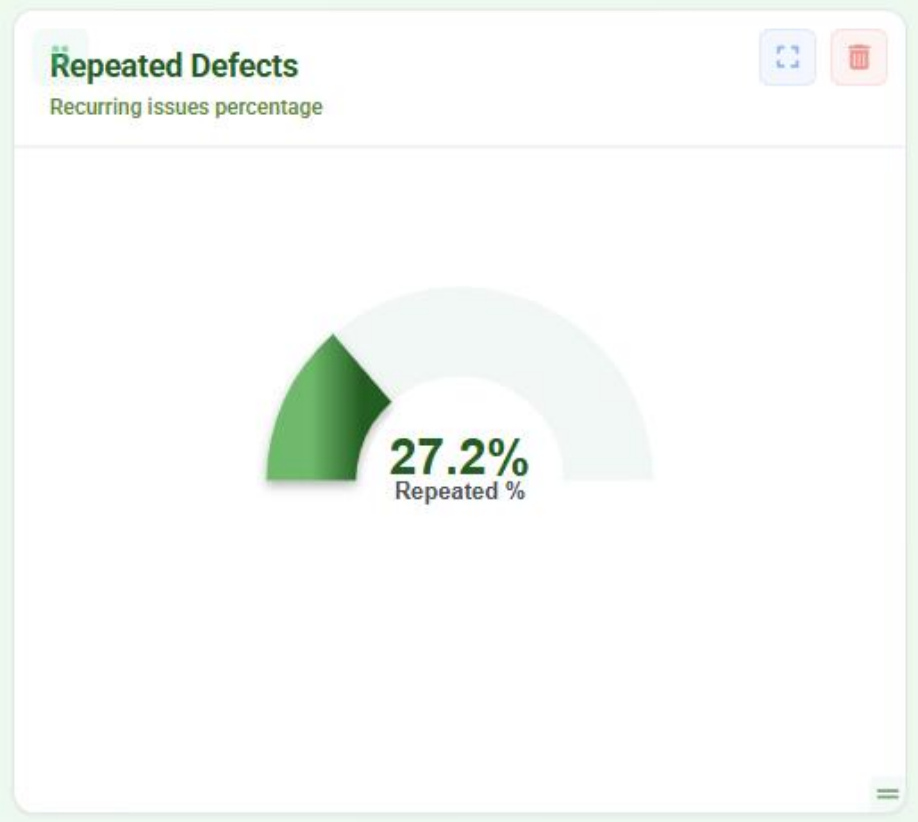
2.10. Vehicle Station Tracking Widget: Render Fix, Performance Optimization, and UX Enhancements



Implemented **rendering fixes and usability improvements** for the **Vehicle Station Tracking** widget, restoring functionality after code merges and ensuring compile compatibility. Enhanced timeline tooltips to display **Station Name, Station Number, and formatted Start/End dates**, providing clearer information when users hover over timeline bars.

Improved **chart readability and responsive behavior** by expanding **X-axis and Y-axis spacing**, adjusting chart dimensions to reduce overlap when displaying many vehicles, and adding **horizontal and vertical scrolling** while keeping vehicle labels visible. These improvements increased the widget's **clarity, usability, and stability** across dashboard views.

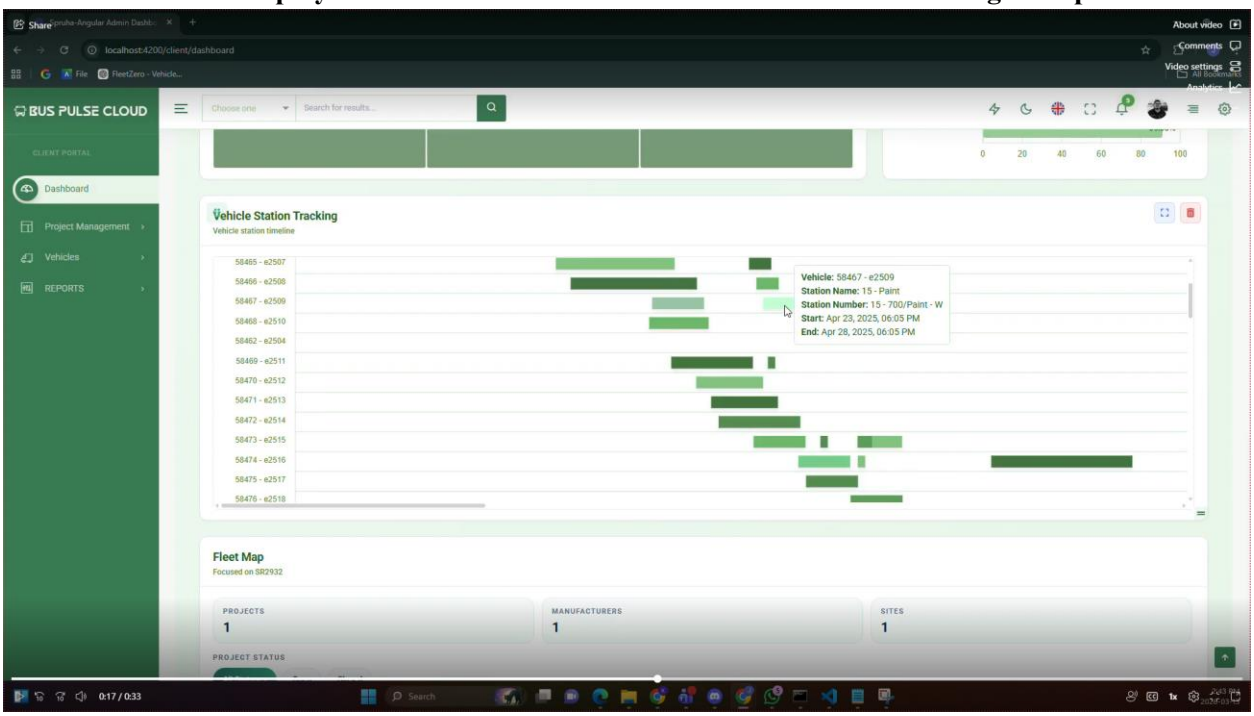
2.11. Fix Client Dashboard widget display logic and Repeated Defects gauge behavior



Improved **Vehicle Station Tracking behavior across both Client and Admin dashboards** by automatically hiding the widget when **All Projects and All Vehicles** are selected, preventing unnecessary or irrelevant timeline data from being displayed.

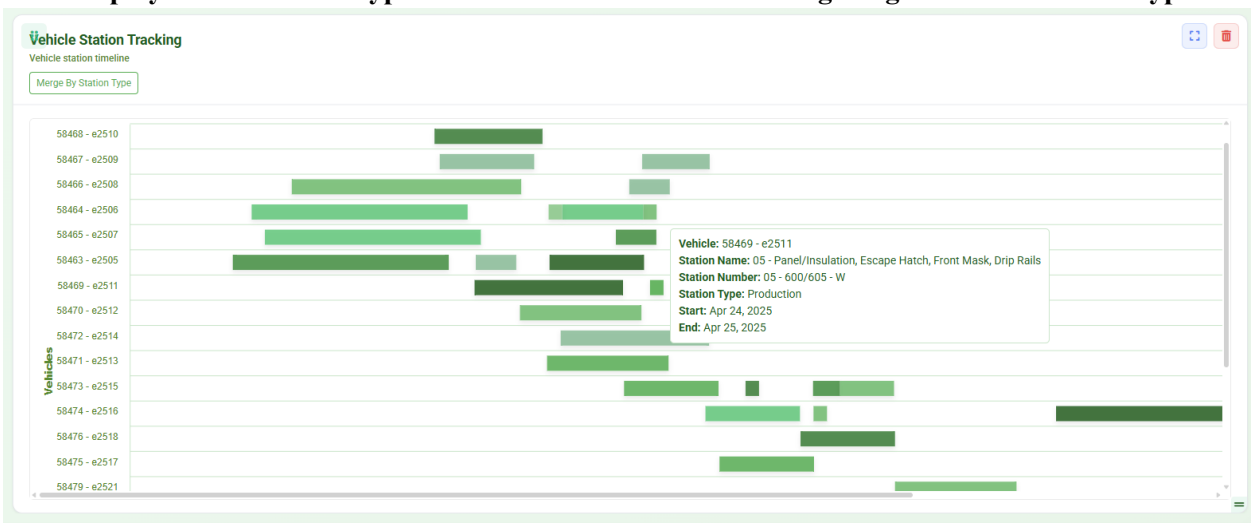
Fixed incorrect **Repeated Defects percentage values** across both dashboards by improving **REST API data processing and percentage normalization**, handling ratio-style values, repeatedTickets, and repeatedPercent, with fallback calculations using **repeatedTickets / totalTickets** and weighted calculations for array responses to ensure accurate defect percentages.

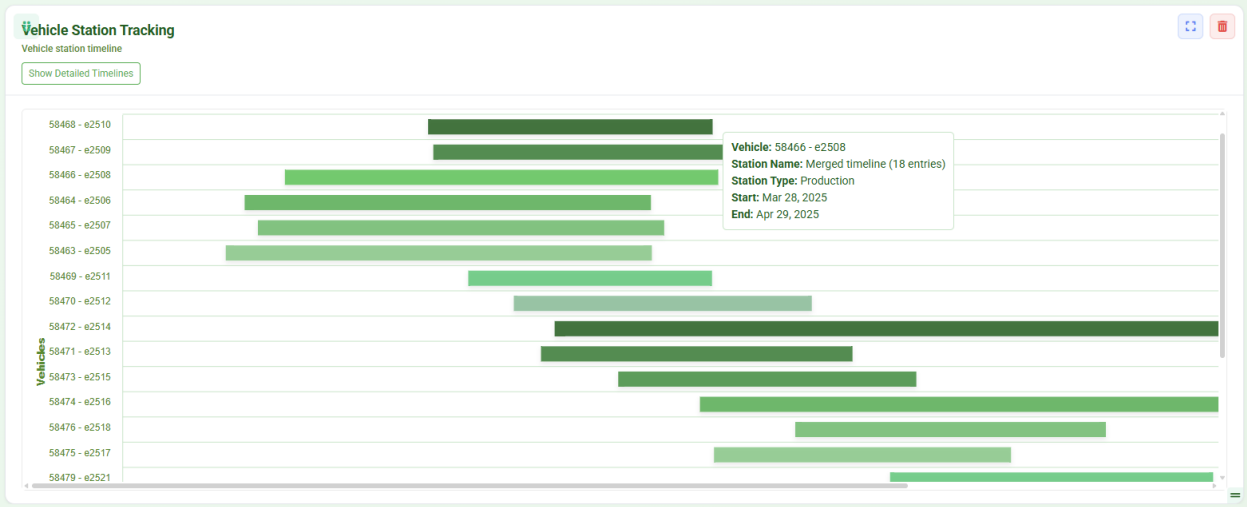
2.12. Remove time display from Start and End dates in Vehicle Station Tracking tooltip



Improved the **Vehicle Station Tracking tooltip** by removing unnecessary time values from the **Start and End dates**, displaying **date-only information** for a cleaner and easier-to-read timeline. Also **merged the latest main branch changes before PR submission** to keep the implementation up to date.

2.13. Display correct stationTypeName in Vehicle Station Tracking widget based on stationTypeId





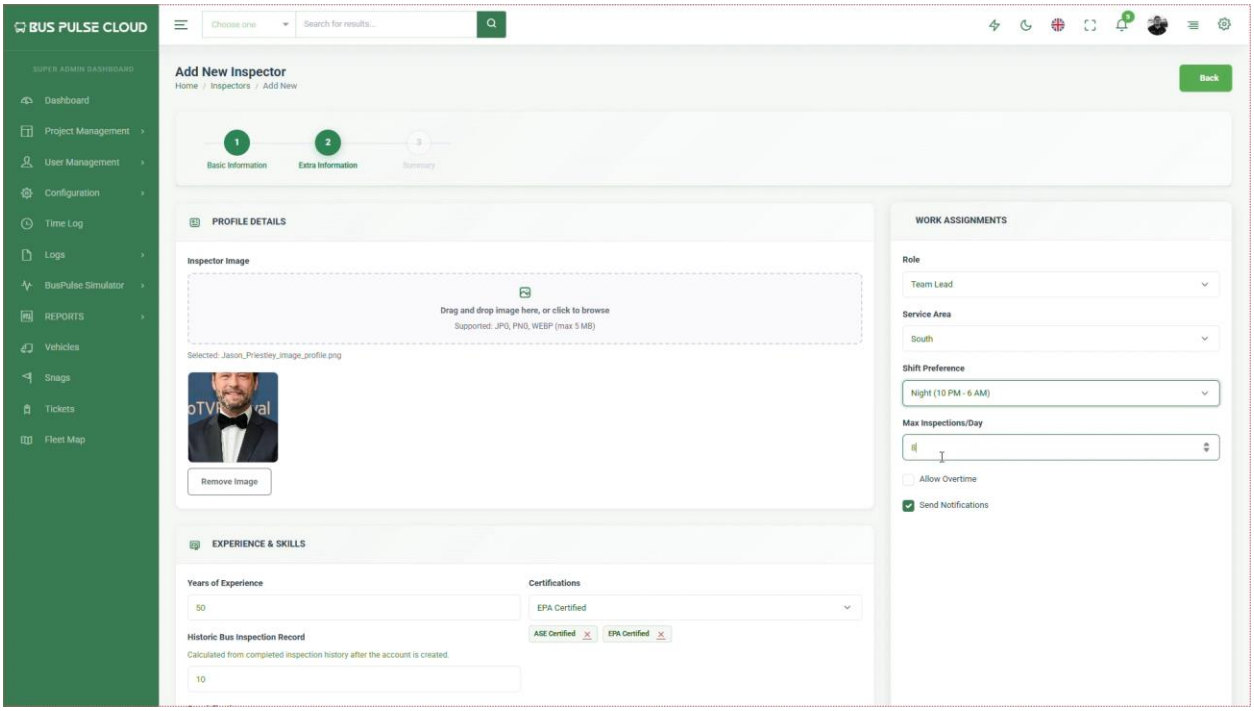
Improved the **Vehicle Station Tracking timeline parsing** for more reliable timestamp handling and added a **toggle between Detailed View and Merged View**, allowing users to switch between individual timeline entries and records grouped by **stationTypeName**. Added **interactive hover details** to display station type and timeline values. Additionally, handled **"None" values returned by the REST API as a valid station type**, ensuring they are correctly processed and displayed as a separate group in the **Merged View**.

2.14. Redesign Projects Comparison by Area Heatmap for Total Defects

Completely redesigned the **Projects Comparison by Area (Total Defects)** visualization to provide a more detailed comparison of defect data across projects and inspection areas. Replaced the previous **station-type comparison structure** with an **area-based heatmap**, displaying projects as rows and inspection areas such as **Engine, Exterior, Function, Interior, Production, Road Test, Roof, Undercarriage, and Water** as columns.

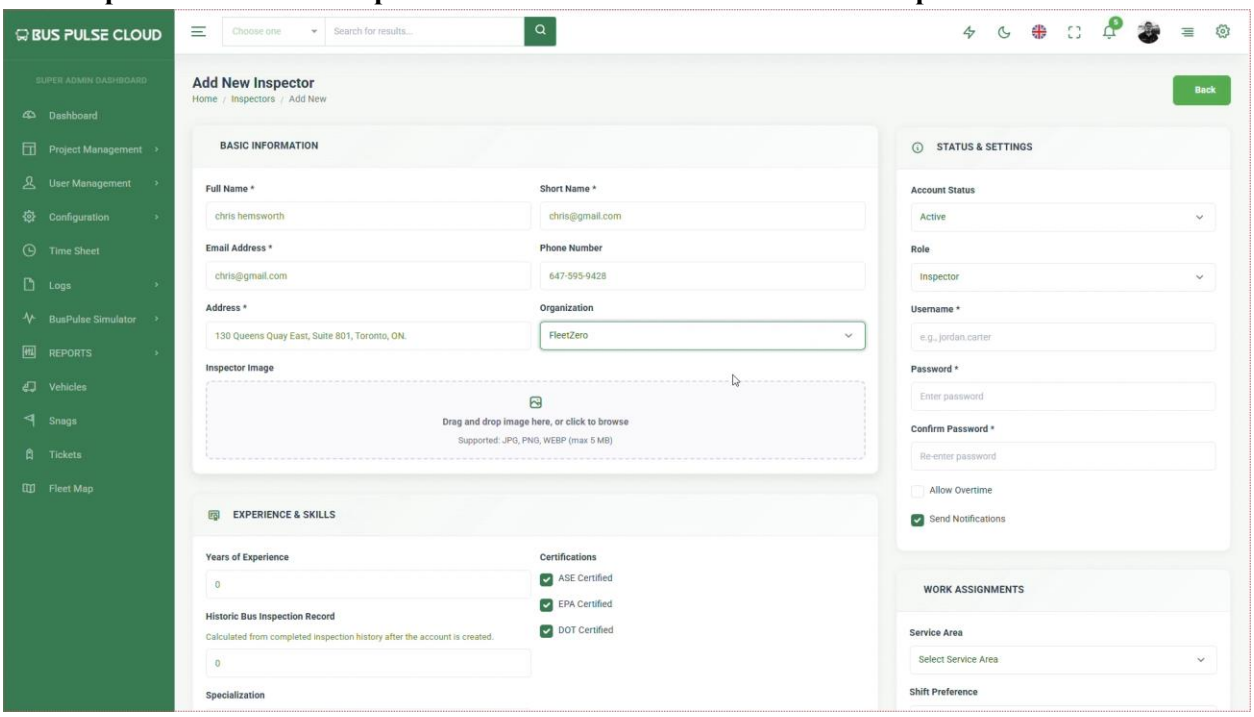
Enhanced the visualization with **dynamic heatmap intensity based on defect counts**, project-type color grouping and legend, visible defect values within each cell, and **interactive hover tooltips** showing the **Project, Project Type, Area, and Defect Count**. Added horizontal scrolling to support the expanded dataset while keeping the comparison readable within the dashboard modal.
The redesigned widget makes it easier to identify **high-defect areas, compare defect distributions between projects, and quickly locate projects or inspection areas requiring attention**, providing significantly more useful operational information than the previous station-type view.

2.15. Develop Inspector Management and Add Inspector Workflow



Developed the **Add New Inspector** workflow with multi-section form handling, validation, image upload/preview, inspector experience and certification fields, specialization, service-area assignment, shift preferences, and inspection-capacity settings. Integrated the workflow into the BusPulse **Inspector Management** interface to support structured inspector profile creation and management.

2.16. Implement Add New Inspector Basic Information and Account Setup

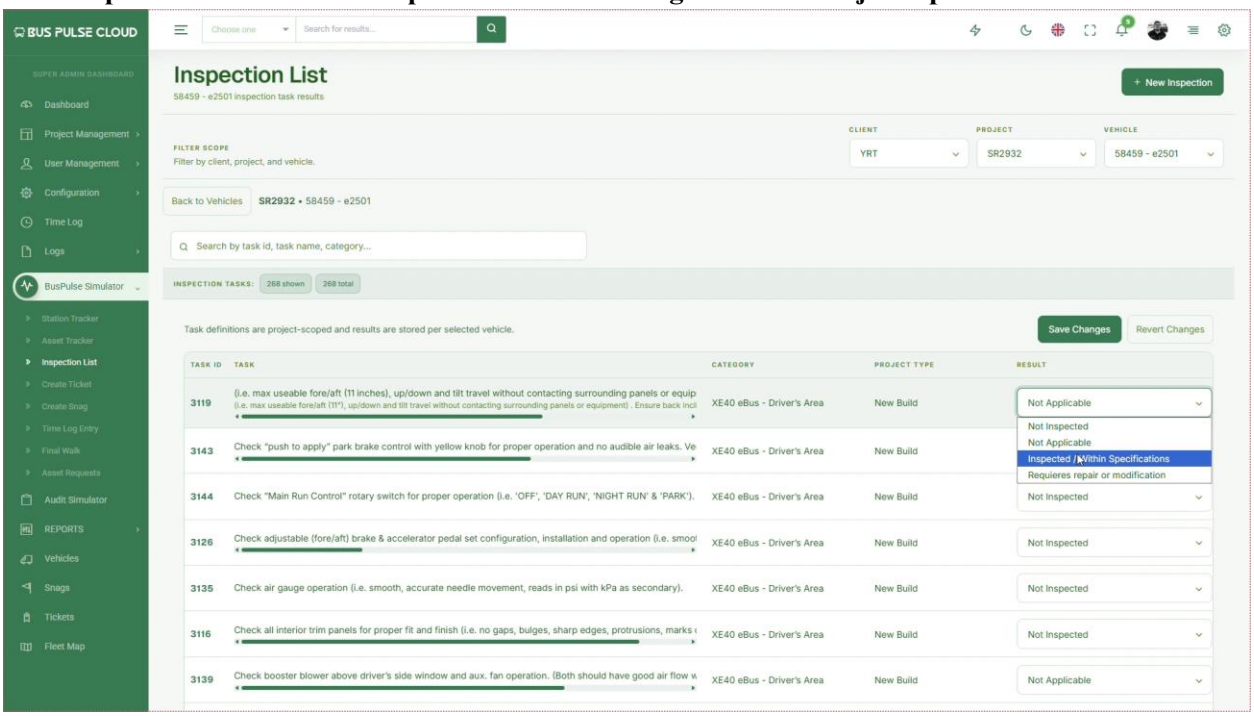


Enhanced the **Inspector Management workflow** to support accessing and editing individual inspector profiles from both **Card and Table views** through the **View Profile** action. Updated the **Inspector Profile and Edit Profile** functionality from the previous **static-data implementation** to support **real REST API integration**, allowing inspector information to be dynamically retrieved and managed for the selected inspector.

Implemented and improved profile fields including **personal information, account status, role, organization, username, contact information, address, profile image, work statistics, experience/skills, and work assignments**. Added **form handling, validation, and update behavior** while maintaining navigation between the **Inspector List, View Profile, and Edit Profile** pages.

The workflow supports selecting a specific inspector from the management interface, loading the corresponding inspector profile by its **inspector ID**, entering **Edit Mode**, updating inspector information, and returning to the **Inspector Management** interface after the operation. This replaced the earlier profile flow that relied on **static inspector data** and established the frontend structure required for **live backend data management**.

2.17. Implement Multi-Level Inspection List API Integration for Project-Specific Vehicle Tasks

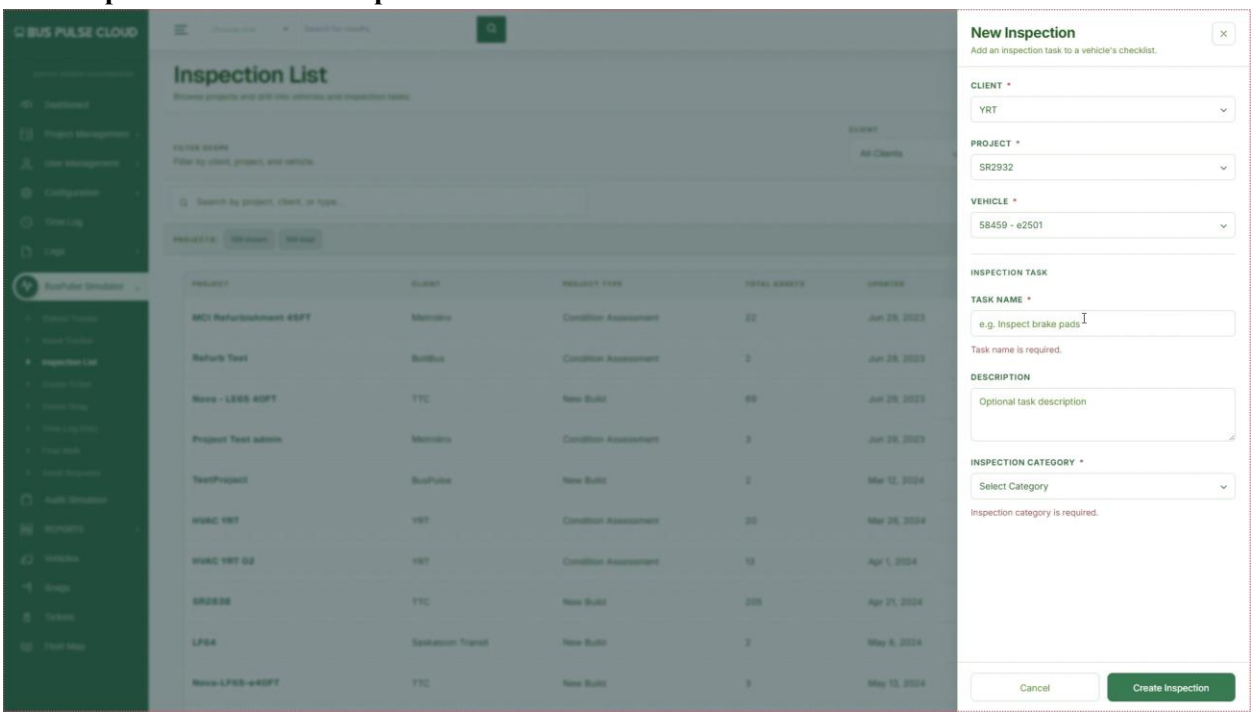


Developed the **Inspection List** workflow to retrieve and display the correct inspection tasks for individual vehicles by integrating multiple related **REST APIs** across the project, vehicle, inspection-task, status, and inspection-result data hierarchy.

Implemented the multi-level data flow required to navigate from a **project to its vehicles and then to the project-scoped inspection tasks for the selected bus**, resolving the relationships between separate API responses before rendering the final Inspection List. The implementation dynamically maintains the selected **client, project, and vehicle context** so the displayed tasks and inspection results correspond to the correct vehicle.

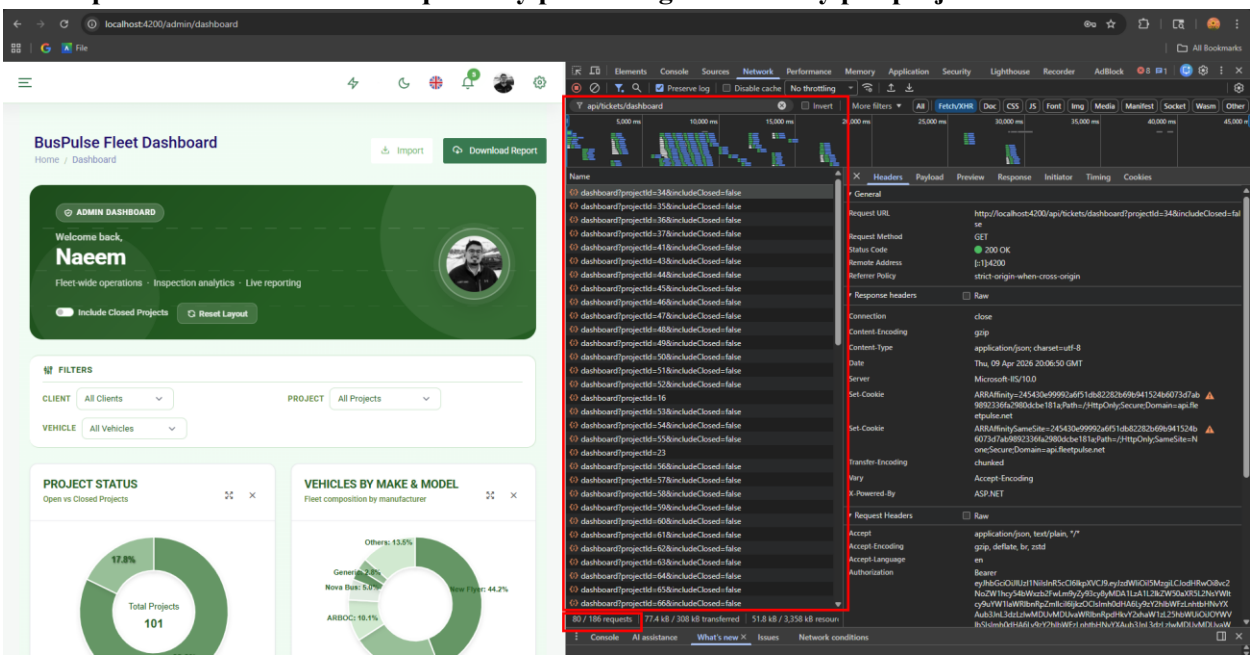
Enhanced the final Inspection List with **Client → Project → Vehicle filtering**, task search, task counts, project-scoped task rendering, inspection-result handling, and save/update functionality. This replaced the earlier limited workflow where the application could not reliably retrieve the correct inspection tasks for each bus because the required information was distributed across multiple API levels.

2.18. Implement Add New Inspection Workflow



Implemented the previously unavailable **"+ New Inspection"** functionality in the Inspection List, transforming the disabled action into a complete workflow for creating a new vehicle inspection.
Developed the multi-step flow for selecting the appropriate **client, project, vehicle, and inspection information**, while maintaining the required relationships between project, vehicle, and inspection data.
Integrated the workflow with the application's **REST APIs** so available options and inspection data are dynamically retrieved based on the user's selections rather than relying on static values.
Added **form validation, dependent selections, inspection configuration, submission handling, and navigation back to the Inspection List**, ensuring newly created inspections remain associated with the correct project and vehicle context.
This enhancement completed an important missing part of the **BusPulse Simulator Inspection List**, allowing users to move from browsing existing inspection tasks to actually **creating a new inspection through the application UI**.

2.19. Optimize dashboard API requests by preventing unnecessary per-project calls

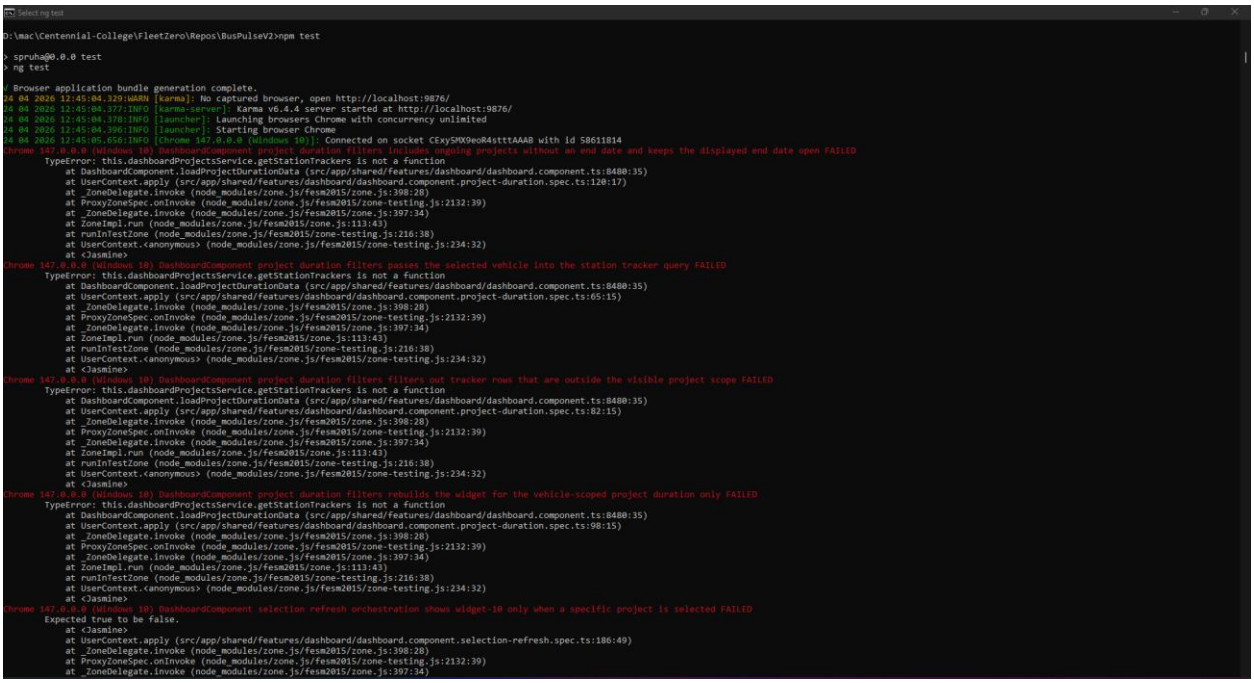


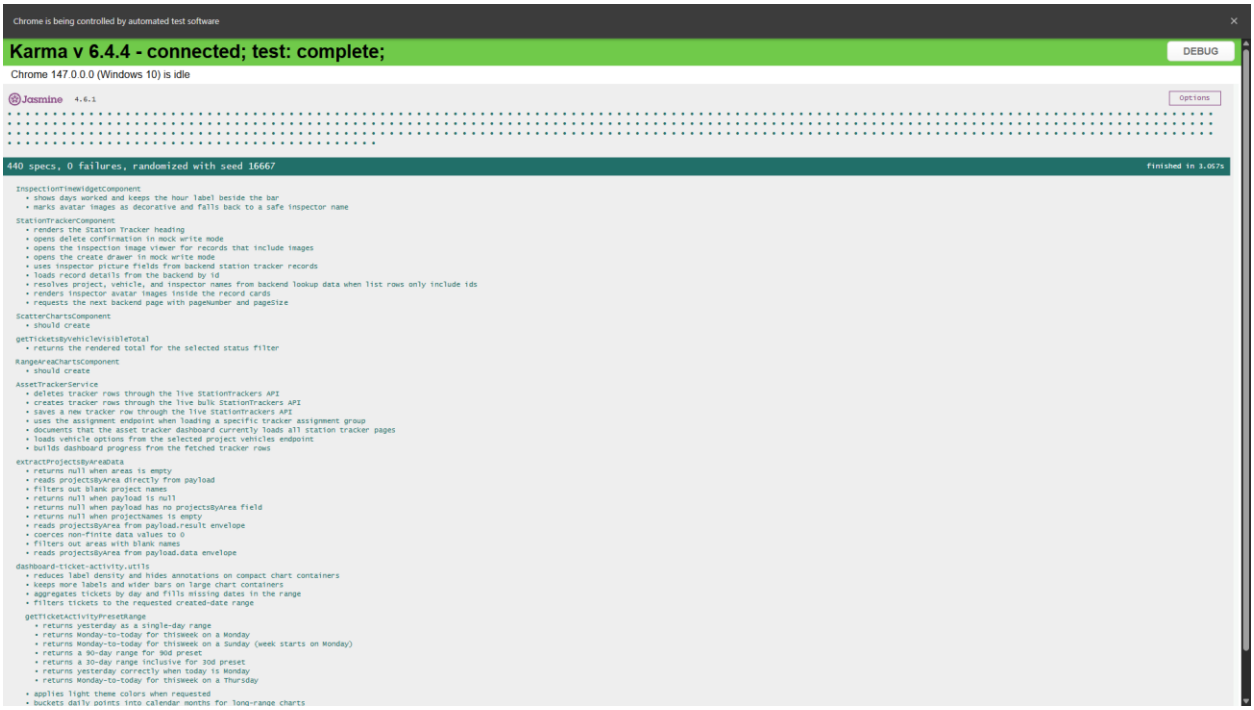
Before the optimization, when the dashboard loaded with the default filters (**All Projects** and **All Vehicles**), it triggered around **80 requests (out of 186 total)** to the /api/tickets/dashboard endpoint. The dashboard was making separate API requests for every project in the background even though no specific project was selected, resulting in unnecessary network traffic and slower page loading.

Updated the dashboard request logic so that when the project filter is set to **All Projects**, it skips the per-project request loop and exits early instead of fetching data for every project individually. Applied the same optimization across related dashboard components to ensure project-specific API calls are only made when a user explicitly selects a project.

These changes eliminated the repeated API requests during the initial dashboard load while preserving the existing behavior for project-specific filtering. The optimization significantly reduced unnecessary network traffic and improved dashboard performance and responsiveness.

2.20. Add Karma/Jasmine unit tests for Station Tracker enhancements





Added new Karma/Jasmine unit tests for the recently implemented Station Tracker enhancements to ensure the feature passes the **CI/CD pipeline** requirements. Updated the corresponding spec files using mocked services and API responses without modifying production code, covering timeline rendering, inspector profile image handling, pagination, API data processing, chart utilities, and dashboard helper functions. The test suite completed successfully with **440 passing, 0 failing** tests and is ready for PR review.